COMMUNITY OVER CODE · GLASGOW 2026

WireGuard for Apache NuttX

A WireGuard VPN that works as a normal network device, `wg0` — on real boards, and inside a real NuttX kernel build.

ESP32-S3 & SPRESENSE
TUNNEL UP OVER REAL WI-FI

Real WireGuard peers
LINUX KERNEL, WINDOWS CLIENT

rv-virt knetnsh64
REAL KERNEL BUILD (BUILD_KERNEL)

NuttX 13.0.1 / master / 12.7.0
APPS V0.1.1 · ONE SOURCE

Satoru Akita · Sony Semiconductor Solutions



Agenda

Seven parts, about thirty minutes.

1. **Overview & background** — ~6 MIN
   Who I am · what NuttX is · how this started
2. **The problem and the idea** — ~4 MIN
   Reaching a device far away · why WireGuard
3. **The plan** — ~1 MIN
   Reuse wireguard-lwip · fit it into NuttX
4. **Version 1: the app** — ~5 MIN
   First tunnel · five hidden bugs · a short video
5. **Version 2: the kernel driver** — ~13 MIN
   Kernel design · two more bugs · testing
6. **Live demo** — ~3 MIN
   StackChan through the tunnel
7. **Summary & next steps** — ~4 MIN
   Upstream plan · three lessons · thanks

Agenda

Seven parts, about thirty minutes.

1. **Overview & background** — ~6 MIN
   Who I am · what NuttX is · how this started
2. **The problem and the idea** — ~4 MIN
   Reaching a device far away · why WireGuard
3. **The plan** — ~1 MIN
   Reuse wireguard-lwip · fit it into NuttX
4. **Version 1: the app** — ~5 MIN
   First tunnel · five hidden bugs · a short video
5. **Version 2: the kernel driver** — ~13 MIN
   Kernel design · two more bugs · testing
6. **Live demo** — ~3 MIN
   StackChan through the tunnel
7. **Summary & next steps** — ~4 MIN
   Upstream plan · three lessons · thanks

About Me

From Japan: I work at Sony, and I build things as a hobby.



Satoru Akita — edge AI engineer at Sony Semiconductor Solutions, working with Apache NuttX from the application side.

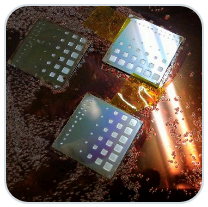


my icon on social media

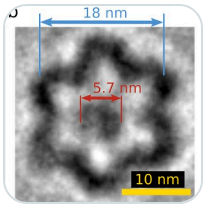
HOBBY ENGINEER



Space



Semiconductors



DNA nanotech



Robots



Linux

- 🎯 Many hackathons
- 🎤 Runs software conferences
- 📖 Wrote a book on open source



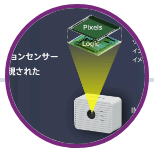
Robotics

Molecular robotics, BIOMOD world champion (2015)



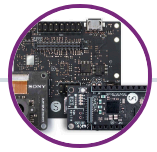
Computer Vision

AI and image processing systems



Edge AI

AITRIOS edge AI cameras



Embedded Systems

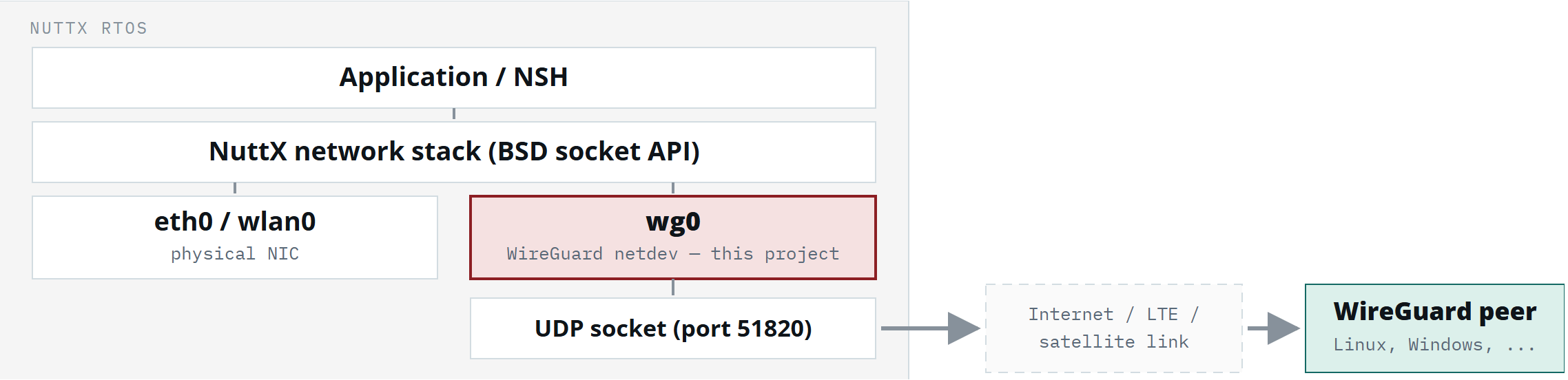
SPRESENSE, embedded app contest — world 3rd

NuttX / WireGuard

This talk

Overview: the project in one slide

This project adds WireGuard to NuttX as a network device.



Packets sent to `wg0` are encrypted and sent out as UDP. UDP that comes in is decrypted and handed over as if it came in on `wg0`. The other side can be any normal WireGuard peer.

TAKEAWAY **Apps see wg0 as a normal network interface. Only the "wire" under it is different: a UDP socket, not a cable.**

How it started

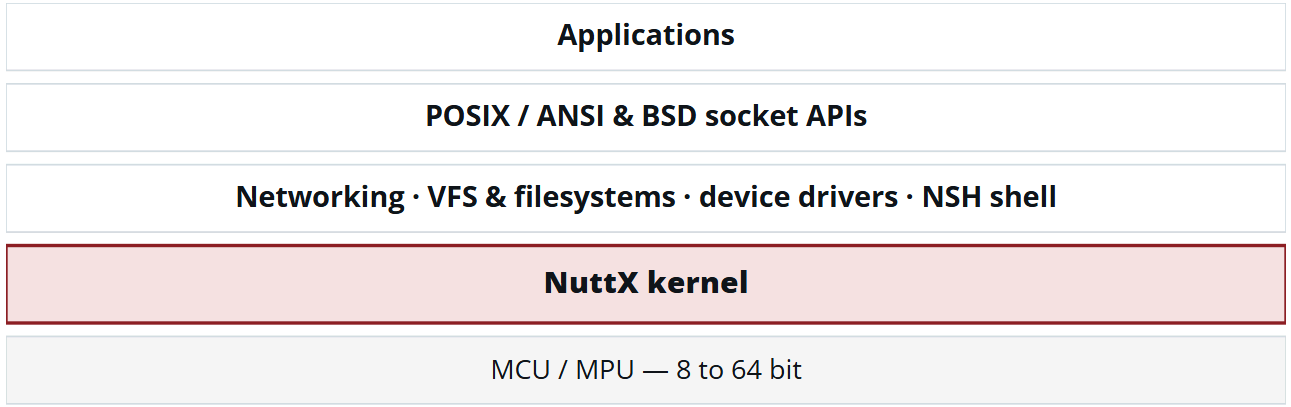
From an idea in February to this talk in October.



TAKEAWAY **The GSoC proposal was not selected, but the work went on. The kernel driver, the harder half, came last.**

Background: the OS behind this work

What is Apache NuttX?



RTOS · POSIX APIs · BSD sockets · Filesystem / VFS · NSH shell · Built-in networking · Device drivers · Apache top-level project

TAKEAWAY **NuttX lets you program small embedded devices the Unix way: POSIX APIs, BSD sockets, files, and devices.**

Background: NuttX and other RTOSes

Where NuttX fits — and where it is used.

RTOS	MAIN FOCUS	TYPICAL USE
FreeRTOS	Small real-time kernel	MCU firmware, control, IoT
Zephyr	All-in-one platform for connected devices	Wearables, industrial & connected IoT
NuttX	**Unix-like embedded OS**	**Networking, file systems, drones, sensing**

> `FreeRTOS/FreeRTOS` 7.8k★ · `zephyrproject-rtos/zephyr` 16.5k★ · `apache/nuttx` 4.1k★
> Stars show how many developers notice a project, not how many devices use it.

ARCHITECTURES
15+ CPU architectures

BOARDS
300+ supported boards

CONFIGS
1,500+ configuration templates

IN PRODUCTION
Sony · PX4 drones · Xiaomi OpenVela (1,000+ SKUs) · Japan's 2024 Moon mission

TAKEAWAY **NuttX has fewer stars than FreeRTOS and Zephyr, but it already runs in real products, from drones to the Moon.**

Sources: nuttx.apache.org · nuttx.apache.org/market-share · docs.px4.io · GitHub star counts, October 2026

History: a long road with NuttX

NuttX at Sony — from audio to edge sensing.

2015 — **Sony audio products**
NuttX inside Sony audio products.

2018–19 — **SPRESENSE / CXD56xx**
Audio, GNSS, camera and multi-core support added to NuttX; shown at Open Source Summit + ELC Europe 2019.

2020 — **Upstream collaboration**
Sony engineers become major NuttX contributors: “A Journey from Fork to Mainline,” NuttX Online Workshop 2020.

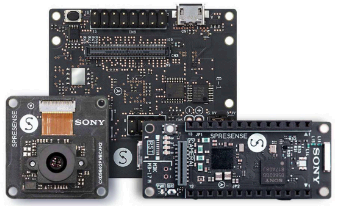
2020s — **Sensing & edge devices**
More camera, GNSS and sensing products.

TAKEAWAY **Sony used NuttX long before edge AI. Years of audio products and SPRESENSE led to sensing and camera systems.**

Sources: M. Ishikawa (Sony), “NuttX for Embedded Linux Developers,” OSS+ELC Europe 2019 · A. Jerpelea (Sony), “A Journey from Fork to Mainline,” NuttX Online Workshop 2020 · nuttx.apache.org/docs (CXD56xx)

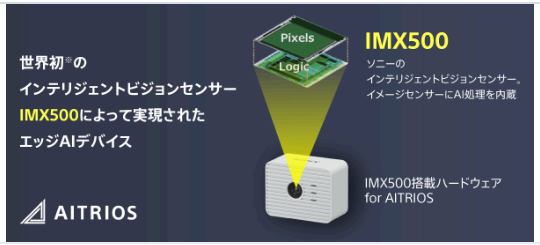
Experience: where I come from

NuttX in SPRESENSE and AITRIOS — my part.



SPRESENSE

NuttX SDK · sensing & camera · GNSS / LTE · prototyping and embedded development.



AITRIOS / edge AI camera

Edge AI · setting up cameras · ESP32 + NuttX · running them on site, not on a desk.

Where I work: application development → system integration → deployment & operation.

TAKEAWAY **My problem was not making NuttX run. It was reaching NuttX devices after they were installed.**

Agenda

Seven parts, about thirty minutes.

1 **Overview & background** ~6 MIN
Who I am · what NuttX is · how this started

2 **The problem and the idea** ~4 MIN
Reaching a device far away · why WireGuard

3 **The plan** ~1 MIN
Reuse wireguard-lwip · fit it into NuttX

4 **Version 1: the app** ~5 MIN
First tunnel · five hidden bugs · a short video

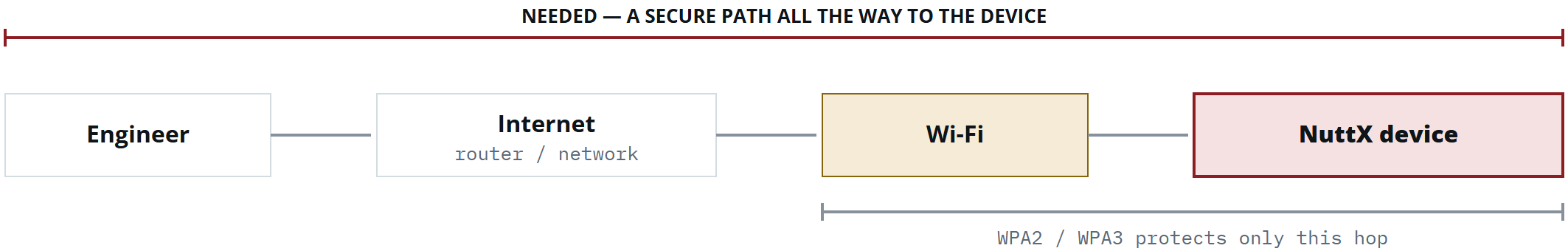
5 **Version 2: the kernel driver** ~13 MIN
Kernel design · two more bugs · testing

6 **Live demo** ~3 MIN
StackChan through the tunnel

7 **Summary & next steps** ~4 MIN
Upstream plan · three lessons · thanks

Problem: Wi-Fi security stops too early

Reaching a device from far away is hard.



OPEN IT TO THE INTERNET

Give the device a public IP and hope.

MAKE YOUR OWN PROTOCOL

Build and maintain your own secure channel.

USE A VENDOR'S CLOUD

Rely on someone else's cloud to reach your own device.

TAKEAWAY **What was missing: a standard, secure path all the way to the NuttX device.**

Options: MQTT, HTTP, router VPN, …

Other ways to reach a device — compared.

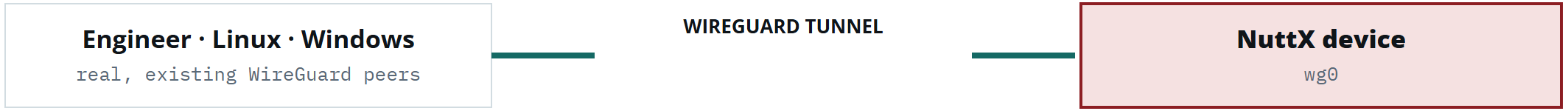
WAY	WHO CONNECTS	ANY SERVICE (TELNET, WEB…)?	SERVER IN THE MIDDLE?	SECURITY ENDS AT	ON NUTTX TODAY
Open a port to the Internet	engineer → device	yes	no	the app (often nothing)	easy, but open to everyone
HTTPS to a cloud API	device → cloud (polling)	no, only your API	yes, a web server	the cloud server	`webclient`, `libcurl4nx` + mbedTLS
MQTT broker (TLS)	device → broker	no, messages only	yes, a broker	the broker	`mqttc`, `paho_mqtt` + mbedTLS
Reverse tunnel / relay (SSH, WebSocket)	device → relay	one port per tunnel	yes, a relay host	the relay host	`dropbear`, `libwebsockets`
VPN on the router	router ↔ office	yes, the whole LAN	no	the router; the LAN hop is open	no change on the device, but you need that router
Carrier closed network (private APN, SIM VPN)	through the carrier	yes	yes, the carrier	the carrier gateway	LTE modem; cellular only
VPN on the device (this talk)	**either side**	yes, any IP service	no, any WireGuard peer	the device itself	`wg0`

TAKEAWAY **MQTT and HTTPS are great for data. To log in and fix a device, end to end, with no server in the middle, the VPN has to be on the device.**

Mesh VPN agents (Tailscale, ZeroTier) are another “VPN on the device” option, but they usually need a bigger OS. NuttX clients: nuttx-apps 13.0.1 `netutils/` (mqttc, paho_mqtt, webclient, libcurl4nx, libwebsockets, dropbear) and `crypto/mbedtls`.

Solution: a VPN on the device

Why WireGuard is a good fit.

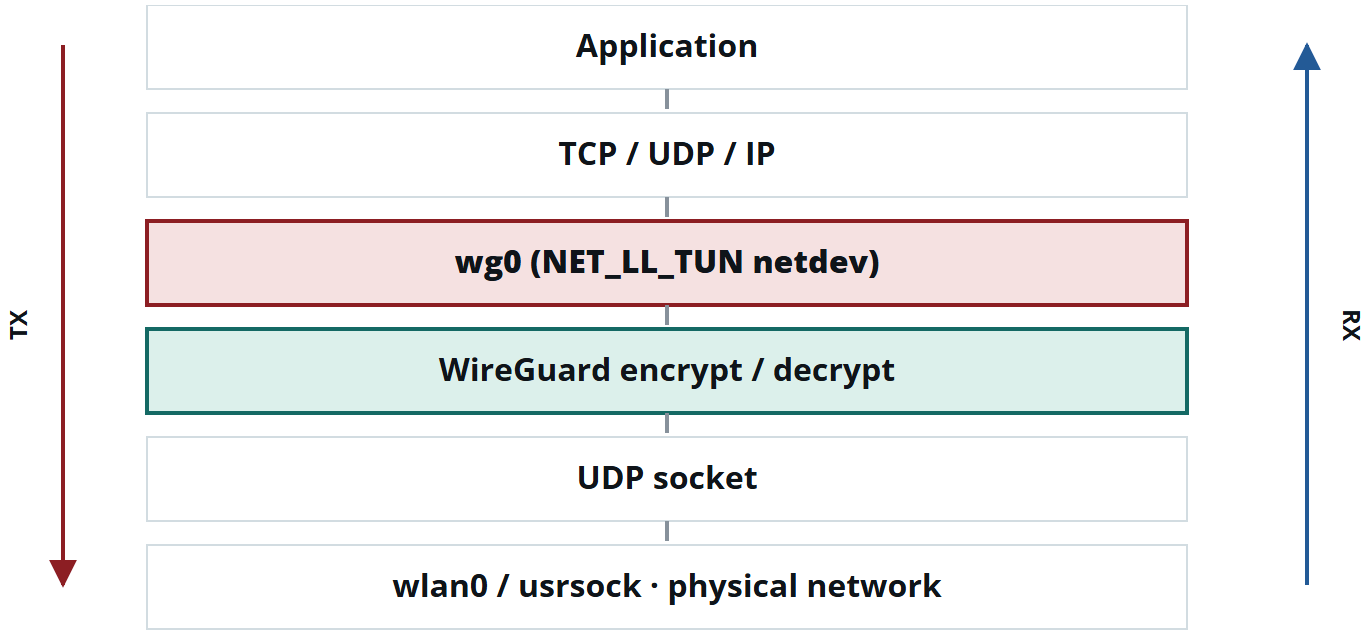


- **Modern IP-layer VPN**
  small and simple: much less code than IPsec or OpenVPN.
- **UDP transport, public-key peers**
  no certificates and no session setup to manage.
- **Already everywhere**
  in the Linux kernel, with an official Windows client. It works with existing peers, not a custom protocol.

TAKEAWAY **WireGuard solves the access problem with a standard protocol, not a homemade security layer or a vendor cloud.**

Architecture: where wg0 sits

WireGuard on the NuttX network stack.



TAKEAWAY **To NuttX, wg0 is a normal network device. Under it, WireGuard turns its packets into encrypted UDP.**

Agenda

Seven parts, about thirty minutes.

1 **Overview & background** ~6 MIN
Who I am · what NuttX is · how this started

2 **The problem and the idea** ~4 MIN
Reaching a device far away · why WireGuard

3 **The plan** ~1 MIN
Reuse wireguard-lwip · fit it into NuttX

4 **Version 1: the app** ~5 MIN
First tunnel · five hidden bugs · a short video

5 **Version 2: the kernel driver** ~13 MIN
Kernel design · two more bugs · testing

6 **Live demo** ~3 MIN
StackChan through the tunnel

7 **Summary & next steps** ~4 MIN
Upstream plan · three lessons · thanks

Porting: four functions to write

Reusing wireguard-lwip without lwIP.

PROTOCOL & CRYPTO — KEEP

Handshake, transport, BLAKE2s, ChaCha20-Poly1305, Curve25519. Portable C, no OS dependency. Used as-is.

PLATFORM HOOKS — ADAPT

Everything OS-specific sits behind `wireguard-platform.h` — four functions: clock, random bytes, TAI64N time, load check.

LWIP NETWORK GLUE — REPLACE

`wireguardif.c`'s lwIP `netif` registration and buffers. NuttX has no lwIP, so this layer is the real porting work.

TAKEAWAY **A clean line between the protocol and the OS let the protocol and crypto move unchanged. Only the OS and network glue had to change.**

Design: one virtual network card

Mapping lwIP onto NuttX netdev.

CONCEPT	LWIP	NUTTX
Network device	struct netif	**struct net_driver_s**
Buffers	pbuf	**iob**
Registration	netif_add()	**netdev_register()**
Start sending	callback	**devif_poll()**

> **`wg0` is a `NET_LL_TUN` netdev, and its wire is a UDP socket.**

TAKEAWAY **Porting meant using NuttX's own building blocks, not bringing lwIP into NuttX.**

Agenda

Seven parts, about thirty minutes.

1 **Overview & background** ~6 MIN
Who I am · what NuttX is · how this started

2 **The problem and the idea** ~4 MIN
Reaching a device far away · why WireGuard

3 **The plan** ~1 MIN
Reuse wireguard-lwip · fit it into NuttX

4 **Version 1: the app** ~5 MIN
First tunnel · five hidden bugs · a short video

5 **Version 2: the kernel driver** ~13 MIN
Kernel design · two more bugs · testing

6 **Live demo** ~3 MIN
StackChan through the tunnel

7 **Summary & next steps** ~4 MIN
Upstream plan · three lessons · thanks

Result: the first working tunnel

A working tunnel is only the beginning.

LAYER	WORK NEEDED	IN THIS PORT
Protocol & crypto	None — portable C, as-is	**3,079 lines, exactly the same as upstream**
OS hooks	Small — four functions	**clock, /dev/urandom, TAI64N, load**
Network glue	The real work	**netdev + UDP socket + RX task**

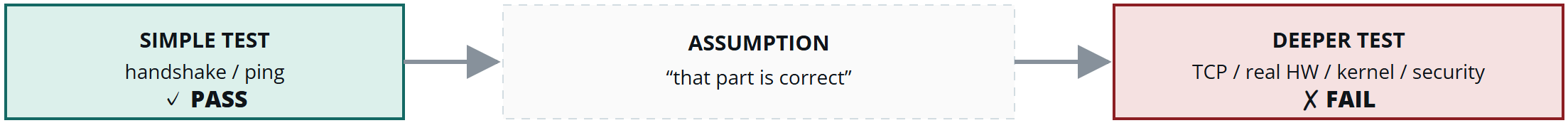
It worked: `wg genkey / set / setconf` at run time. No rebuild to change keys or peers.

And this is where the interesting part starts.

TAKEAWAY **Getting a tunnel up was the easy step. Almost everything interesting came after it already "worked."**

The Testing Pattern

Simple tests pass. Deeper tests fail.



A simple test passes. A deeper one fails.

Because the simple test passed, you trust that part and spend hours looking somewhere else. This happened again and again in this project, and not only in the apps version.

Bug 1 / 7

A return code of 0 that did nothing.

`setsockopt(SO_RCVTIMEO)` returns **0 (success).**

But `recvfrom()` still **blocks forever.** The timeout was accepted, then ignored.

A perfect deadlock:

timer never fires → no handshake is sent → the peer stays silent → `recvfrom()` never returns.

TAKEAWAY **setsockopt() returning 0 does not prove the option works. An unsupported option can show up as a deadlock, not as a clear error.**

Bug 2 / 7

Created successfully. Did not survive.

The RX thread was started with `pthread_create()` + `pthread_detach()`.

When the `wg` command exited, the thread **stopped** (not even visible in `ps`).

A `usleep()` right after create made the thread's `printf` appear, so it **was** created.

My mistake: I thought `detach` meant "lives longer than the command." It only controls **when the thread's memory is freed**, not whether it keeps running after the command exits.

TAKEAWAY **pthread_detach() decides when memory is freed, not how long the thread lives. A detached thread still dies with the task group that started it.**

Bug 3 / 7 · my favorite

ping works. TCP dies.

Same tunnel: `ping` is **0% loss.** A telnet 3-way handshake even **completes**, then the connection dies.

```
wg_encrypt_and_send: psock_sendto
ret=-1 errno=9  EBADF
```

The sends that **worked** ran inside the `wg_rx` task. Sends from another task got EBADF.

Cause: an fd belongs to one **task group**; it is not global.

Fix: use the internal API without fds: `psock_*()` on a `struct socket`, which any task can use.

TAKEAWAY **In NuttX, a file descriptor belongs to one task group. Another task cannot use it, even though it points to the same socket.**

Bug 4 / 7

“No real impact” was 10×.

The fix without fds also broke `poll()`. Quick fix: check every 50 ms.

I thought: “it only wakes the CPU when idle.” Wrong: each packet waited up to 50 ms, and that made it **10× slower.**

Fix: hook `psock_poll()` with a callback and a semaphore (a real blocking wait).

TAKEAWAY **The 50 ms polling kept things correct but made it about 10× slower. A quick fix you never measure can look free and cost 10×.**

Bug 5 / 7

sim passes. Real hardware is different.

Saving the config with `wg showconf > /data/wg0.conf` froze the board, only on **real hardware with SPIFFS**, never in sim. Fixed with `wg saveconf`, which writes the file itself.

The `wg_rx` task calls `ipv4_input()` on **its own stack**, so the whole TCP call chain runs on it, plus encrypting the reply.

Default **3072 B** was a number nobody had measured. On the board it was **94% used, 176 B left** (`CONFIG_STACK_COLORATION` + `ps`).

TAKEAWAY **A default stack size is only a guess until you measure it on real hardware. Sim's big stacks hide the problem.**

Version 1 demo: the first video

Version one works: telnet and a web page through the tunnel.

```
1  telnet 10.10.0.2   # log in through wg0
2  uname -a, ifconfig
3  webserver &
4  browser: http://10.10.0.2/
```

ESP32-S3 DevKit, apps version (v0.1.1).

USB unplugged: the board runs on a power adapter only.

The other end: the official Windows WireGuard client, over home Wi-Fi.

`Video: youtu.be/1kyX2av5WG4`

TAKEAWAY **Version one was enough for a demo. Next, I made it a real NuttX driver.**

Agenda

Seven parts, about thirty minutes.

1 **Overview & background** ~6 MIN
Who I am · what NuttX is · how this started

2 **The problem and the idea** ~4 MIN
Reaching a device far away · why WireGuard

3 **The plan** ~1 MIN
Reuse wireguard-lwip · fit it into NuttX

4 **Version 1: the app** ~5 MIN
First tunnel · five hidden bugs · a short video

5 **Version 2: the kernel driver** ~13 MIN
Kernel design · two more bugs · testing

6 **Live demo** ~3 MIN
StackChan through the tunnel

7 **Summary & next steps** ~4 MIN
Upstream plan · three lessons · thanks

Part 5 · Going deeper

Into the kernel.

The limit of version 1

It lives in `apps/` and uses `psock_*` internals. Fine for a FLAT build. But it can never be a proper in-tree driver, and it does not work in **PROTECTED / KERNEL** builds, where apps and kernel are kept apart.

The move

Put the device **in the kernel** (`drivers/net/wireguard/`) and control it from a small user-space `wg` command with **ioctl**.

TAKEAWAY **An app-level port proves the protocol works. Only a kernel driver is a proper device that also works in PROTECTED and KERNEL builds.**

The Kernel Design

A kernel netdev, an ioctl ABI, a write-only key.

Kernel netdev

`netdev_lowerhalf` with its own UDP socket, receive thread, timers and cookies. The driver owns the socket and the thread, so they no longer die with the command that started them.

Simple ioctl ABI with no pointers

`SIOCS/GWGIF`, `SIOCS/D/GWGPEER`. No pointers, because in PROTECTED / KERNEL builds the kernel copies the caller's struct as it is.

Private key is write-only

the get ioctl never returns the private key.

Kernel crypto is NuttX's own

BLAKE2s, ChaCha20-Poly1305, Curve25519 from `crypto/`. No copied code in the kernel. The only copied file, an MIT-licensed X25519, is in the user-space command for offline `genkey`. Using the kernel's Curve25519 through cryptodev would pull in the whole software cipher set for one multiply.

TAKEAWAY **The socket, the thread and the no-pointer ABI follow one rule: don't depend on code that can exit, or on memory the kernel can't reach.**

Two more bugs: same shape

Deeper level, same kind of bug.

A bug in NuttX's own crypto

`chacha20poly1305` put the counter in the wrong nonce bytes. Counter 0 matches, so **the handshake succeeds and every data packet after the first fails.** Nothing in NuttX had ever used it. Two-line fix, going upstream as a separate `crypto:` PR.

A BUILD_KERNEL-only crash

`wg_set_if()` copied the peers into a **6 KB array on the stack**; the kernel stack is 3 KB → overflow → heap corruption → panic. **sim (FLAT, big stacks) never hit it.** Moved to the heap.

TAKEAWAY **A working handshake does not prove the data path or the kernel code is right. Both bugs only showed up after the handshake, in a real kernel build.**

Testing: simulator, QEMU, real boards

Seven places it must work.

WHERE	VERSION	WHAT I CHECKED
sim:wireguard	kernel driver	builds with no errors (the config CI builds)
rv-virt knetnsh64	**kernel · BUILD_KERNEL**	**virtio-net tunnel to Linux; `wg` is a separate program that calls in through syscalls**
rv-virt pnsh64	**kernel · BUILD_PROTECTED**	**tunnel with the MPU split on: `wg` in `nuttx_user`, every ioctl crosses the boundary; tunnel works, but 2 of 8 repeated runs hit a fault (open)**
rv-virt knetnsh64_smp	**kernel · SMP, 4 CPUs**	**heavy traffic while changing the config and doing `down` / `up`; no fault, tunnel came back**
SPRESENSE, measured	**kernel · Cortex-M4F**	**under load and 40 `down` / `up` cycles: heap, buffers and stacks exactly the same; receive thread uses 1472 B of 6096**
ESP32-S3 + SPRESENSE	**kernel · real Wi-Fi**	**tunnel to the Windows client: ESP32 native 4/4, SPRESENSE via usrsock 6/6**
same two boards	apps v0.1.1	Linux & Windows peers; telnet, HTTP, 7 MB, rekey, power-cycle

> **Both boards, PROTECTED and KERNEL, many CPUs: measured, not guessed.**

TAKEAWAY **A tunnel in sim is not enough. Test real builds, many CPUs, and real hardware.**

Hardware tests used the version before the final rebase onto upstream; KERNEL/SMP tunnels were run again on the rebased version.

Testing: what ping misses

A working tunnel does not prove enough.

PROBLEM	WOULD PING NOTICE?	WHAT I MEASURED
The same private key on every boot: the xorshift128 `/dev/urandom` backend can be seeded from build-time values	no, every normal test still passes	**3 restarts → 3 different keys; a basic check of the entropy pool path, not proof of good randomness**
Session keys left in memory after down	no	**read the running memory: transport keys 32/32 bytes set while up → all zero after `down`; exactly one copy of the static key, before and after**
A leak that only shows after hours	no	**151 forced rekeys past the key lifetime, 10 endpoint changes, 100 `down` / `up`: no buffers lost, allocations back to the boot count exactly, RX-thread stack peak 2.5 KB and not growing**

> **Each of these passes every normal test while being broken.**

TAKEAWAY **A working tunnel does not prove unique keys, cleared secrets, or stable memory. Each needs its own check.**

Design choice: trust the clock?

The timestamp problem has no easy answer.

A peer drops any handshake whose timestamp is not newer than the newest it has seen. So the device must always send a newer one, **even after a reboot**.

OPTION	FLASH WRITES	WORKS WITHOUT RTC	RESULT
Real-time clock + highest value this boot	none	no	**chosen**
Save every timestamp	every handshake	yes	wears out flash, slows the handshake
Reserve a range in flash	one per N	yes	needs a storage guarantee NuttX does not have
Wait until it catches up	none	no	you wait as long as it last ran

Same board, one change, nothing sent to it: **clock unset → no handshake in 75 s**; **clock set → 4.1 s**.

> **Used the same rule as Linux and wireguard-go, and made the case it misses print a warning.**

TAKEAWAY **No clock, no newer timestamp: a device may be unable to connect out and still pass a ping test**

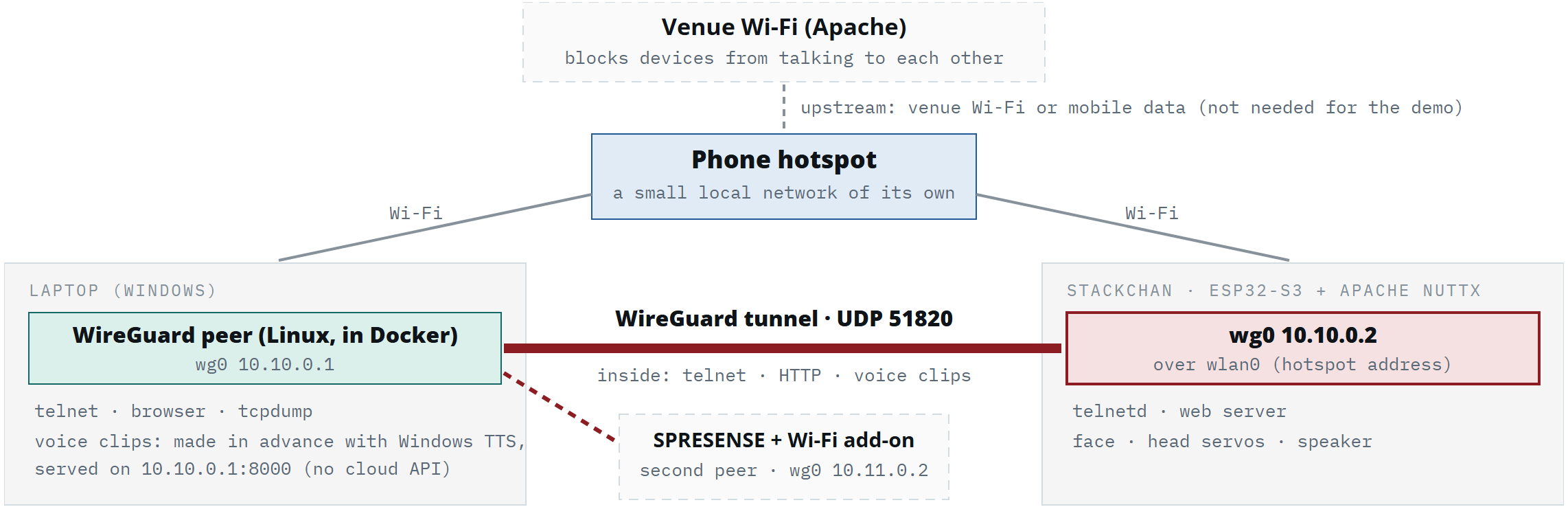
Agenda

Seven parts, about thirty minutes.

1. **Overview & background** — ~6 MIN
   Who I am · what NuttX is · how this started
2. **The problem and the idea** — ~4 MIN
   Reaching a device far away · why WireGuard
3. **The plan** — ~1 MIN
   Reuse wireguard-lwip · fit it into NuttX
4. **Version 1: the app** — ~5 MIN
   First tunnel · five hidden bugs · a short video
5. **Version 2: the kernel driver** — ~13 MIN
   Kernel design · two more bugs · testing
6. **Live demo** — ~3 MIN
   StackChan through the tunnel
7. **Summary & next steps** — ~4 MIN
   Upstream plan · three lessons · thanks

Demo setup: the network

One tunnel, on a phone's local network.



TAKEAWAY **The phone and the venue only ever see encrypted UDP, with no server in the middle and no cloud.**

Demo: live on the StackChan

Log in through the tunnel, and talk to the robot.

```
wg show           # handshake, bytes
telnet 10.10.0.2   # log in
ifconfig          # wlan0 and wg0
stackchan face happy
stackchan say hello.wav
tcpdump: outside vs. inside
wg show           # bytes went up
```

Outside the tunnel (the phone's network): only encrypted UDP on port 51820.

Inside the tunnel (wg0): the telnet text you just typed.

Backup video (earlier demo): youtu.be/1kyX2av5WG4

Agenda

Seven parts, about thirty minutes.

1 **Overview & background** — ~6 MIN
Who I am · what NuttX is · how this started

2 **The problem and the idea** — ~4 MIN
Reaching a device far away · why WireGuard

3 **The plan** — ~1 MIN
Reuse wireguard-lwip · fit it into NuttX

4 **Version 1: the app** — ~5 MIN
First tunnel · five hidden bugs · a short video

5 **Version 2: the kernel driver** — ~13 MIN
Kernel design · two more bugs · testing

6 **Live demo** — ~3 MIN
StackChan through the tunnel

7 **Summary & next steps** — ~4 MIN
Upstream plan · three lessons · thanks

Running it for real

From a demo to something you can run.

	BEFORE	AFTER
Keys	built into Kconfig; rebuild to change	**genkey / set / setconf at run time; never in the firmware image**
When it stops	guessing	**check ROUTER / LAN / TUN / TCP every minute to find which part failed**

> **Four separate checks answer one question: when the tunnel goes quiet, is it my side?**

TAKEAWAY **Run-time keys and per-layer checks turn "the tunnel is down" into "the router side is down." That is the difference between a demo and something you can leave running.**

Other CPUs: beyond the first board

Did the “four functions” bet work?

The apps (FLAT) version ran on four CPU types with **no code change**:

TARGET	CODE CHANGE	RESULT
x86_64 (sim)	none	**tunnel**
ARM Cortex-A7 (QEMU)	none	**tunnel**
Xtensa LX7 (ESP32-S3)	none	**tunnel over real Wi-Fi**
ARM Cortex-M4F (SPRESENSE)	none	**tunnel over real Wi-Fi (GS2200M)**

TAKEAWAY **Four small OS functions let one source run on four CPU types, unchanged. The bet paid off.**

Checked with the apps (FLAT) version; the kernel driver was also checked on sim, rv-virt (BUILD_KERNEL) and both boards over real Wi-Fi (ESP32-S3 native, SPRESENSE over usrsock).

Giving back: sending it to NuttX

The Apache Way: small steps, in the open.

- **A small `crypto:` PR first**

  the ChaCha20-Poly1305 nonce fix, which helps NuttX on its own.

- **Then the driver PR**

  `drivers/net/wireguard` — the kernel device, ABI, and its documentation.

- **Then the command PR**

  `apps/system/wg` — the command that talks to the driver.

- **Design ready for `dev@nuttx.apache.org`**

  passes checkpatch / nxstyle, split into PRs, licenses checked. Community over code.

TAKEAWAY **Sending the crypto fix as its own PR lets NuttX take it even if the driver is not accepted.**

Status: the work is split into small PRs that pass checkpatch and build one commit at a time; the dev@ discussion and the PRs come next.

Summary

What I built, and three things to remember.

BUILT `wg0`: **a WireGuard network device in the NuttX kernel. It works on ESP32-S3 and SPRESENSE, talks to standard WireGuard peers, and is split into small PRs for upstream.**

– Simple tests pass, deep tests fail

test deeply (long data runs, real hardware, a real kernel build), not just “the handshake worked.”

– A clean OS boundary pays off

four CPU types, three NuttX versions, no changes.

– Porting to a project can make the project better

two hidden NuttX bugs found on the way: its ChaCha20-Poly1305 nonce, and a boot bug that stops a clean release from reaching the shell on this board.

Thanks: the people behind this talk

Many people helped me get here.

Alan

for telling me about this conference and encouraging me to submit a talk.

The Apache NuttX community

for talking with me about my GSoC proposal, and for encouraging me to keep going.

The Apache Software Foundation

and the Community Over Code organizers and volunteers, for making this event happen.

Sony's NuttX developers, past and present

who brought NuttX into our products and gave so much back to the project.

COMMUNITY OVER CODE · GLASGOW 2026

Thank you

Questions welcome — the kernel driver, the bugs, the ioctl ABI, or the hardware.

youtu.be/1kyX2av5WG4
DEMO

apache/nuttx · apache/nuttx-apps
GOING UPSTREAM

Satoru Akita · Sony Semiconductor Solutions

